Instituto Federal Goiano — Campus Morrinhos
Curso de Bacharelado em Sistemas de Informação

Contas a Pagar e a Receber

Interface web em React e TypeScript, integrada a uma API REST

Autor: Felipe Montalvão Rodrigues

Front-end: React 19 + TypeScript + CSS puro (Vite)

Back-end consumido: Spring Boot + MySQL

Referência: SHIRAISHI, G.; CÔRTES, P. *Delphi 4 — Contas a Pagar e a Receber*

Morrinhos, outubro de 2026

Sumário

1. Design

- 1.1 Do protótipo à tela
- 1.2 A linguagem visual e como ela foi reproduzida em CSS
- 1.3 Estados, foco e teclado
- 1.4 Layout adaptável

2. Implementação

- 2.1 Organização das pastas
- 2.2 TypeScript sem `any`
- 2.3 Um formulário que se monta a partir de um JSON
- 2.4 Duas telas que servem quatro cadastros
- 2.5 Estado: o que a tela guarda e o que ela busca
- 2.6 Testes

3. Integração com a API

- 3.1 O caminho de uma requisição
- 3.2 O cliente HTTP
- 3.3 O padrão Adapter
- 3.4 O hook de carregamento
- 3.5 O problema que mais deu trabalho: PUT substitui a coleção
- 3.6 Regras de negócio na interface
- 3.7 Como a integração foi validada

4. Uso de inteligência artificial

- 4.1 Como a IA foi usada
- 4.2 O que ela acertou
- 4.3 O que ela errou, e como os erros apareceram
- 4.4 O método que sobrou

Anexo A — Como executar

Anexo B — Telas e endpoints

1. Design

1.1 Do protótipo à tela

O ponto de partida foi um protótipo no Figma que reproduz a aparência do sistema original do livro — um aplicativo Delphi de 1998, com barra lateral em forma de pastas de arquivo, faixas azul-marinho e controles em relevo. A decisão de projeto, tomada logo no começo, foi **não modernizar**: o trabalho é implementar aquele protótipo, não propor outro.

Isso tem uma consequência prática que atravessa o relatório inteiro. A tela fala a língua do livro — "Registro", "Cliente", "CGC", "Nova Conta" — enquanto a API fala a língua do contrato REST — `id`, `nome`, `documento`, `codigo`. As duas línguas não se encontram por acaso: existe uma camada só para traduzir uma na outra, descrita na seção 3.3.

Os elementos gráficos não foram redesenhados à mão: os 163 vetores SVG do protótipo (as pastas marrons, as etiquetas azuis, os ícones) foram exportados e guardados em `public/assets/`, com as dimensões originais anotadas em `src/assets.json`. Um script, `scripts/verify-assets.mjs`, confere que nenhum deles sumiu ou mudou de tamanho — é o que impede o layout de escorregar sem ninguém perceber.

1.2 A linguagem visual e como ela foi reproduzida em CSS

Todo o estilo está em um único arquivo, `src/styles.css`, com 210 linhas. Não há Tailwind nem biblioteca de componentes: o enunciado pede CSS puro, e o resultado é que cada efeito visível na tela corresponde a uma regra que dá para apontar com o dedo.

As cores e os dois relevos característicos do Delphi são variáveis CSS declaradas uma vez:

```
:root {
  --navy: #000080;        /* faixas de título */
  --sidebar: #173652;     /* fundo da barra lateral */
  --workspace: #d5d5d5;   /* área de trabalho */
  --folder: #b98f59;      /* marrom das pastas */
  --raised: 1px 2px 2px #00000040, inset 1px 1px 0 #ffffff6b,
            inset -1px -1px 0 #00000052;
  --sunken: inset 2px 2px 3px #00000055, inset -1px -1px 0 #ffffff8c;
}
```

`--raised` e `--sunken` são o miolo da aparência. O relevo do Delphi é uma luz vindo de cima e à esquerda: uma borda interna clara em cima, uma escura embaixo, e uma sombra projetada fora. O botão pressionado é o mesmo efeito invertido — por isso `--sunken` existe como um par de `--raised`, e trocar um pelo outro no `:active` basta para o botão "afundar".

Falta um detalhe que o CSS sozinho não dá: no Delphi, o texto do botão pressionado também se desloca. Daí a regra `.delphi-button:active span { transform: translate(2px, 2px); }` — dois pixels para baixo e para a direita, que é o que o protótipo mostra.

1.3 Estados, foco e teclado

Cada controle tem quatro estados: normal, *hover*, pressionado e foco por teclado. O último é o que costuma ficar de fora, e foi tratado de propósito: `:focus-visible` desenha um contorno azul de 2 px afastado 3 px do controle, visível sobre o cinza e sobre o azul-marinho.

Os modais usam o elemento `<dialog>` nativo do HTML em vez de uma `div` com `position: fixed`. A escolha foi deliberada: o `<dialog>` prende o foco do teclado dentro de si, fecha com `Esc` e devolve o foco ao botão de origem sem que seja preciso escrever uma linha de JavaScript para isso. Um modal caseiro exigiria reimplementar as três coisas — e normalmente só a primeira é lembrada.

Quando o usuário troca de tela, o foco vai para o título `<h1>` da tela nova. Sem isso, quem navega por teclado continuaria no mesmo ponto da barra lateral, sem saber que o conteúdo mudou.

1.4 Layout adaptável

O protótipo foi desenhado para uma janela larga. Em telas menores, a barra lateral recolhe e as tabelas ganham rolagem interna, em vez de empurrar a página para os lados. A estrutura principal é uma grade de duas colunas (`grid-template-columns: 240px minmax(0, 1fr)`); o `minmax(0, 1fr)` é o que impede uma tabela larga de esticar a coluna além da janela — um detalhe pequeno que, sem ele, quebra o layout inteiro.

2. Implementação

2.1 Organização das pastas

São 48 arquivos TypeScript em `src/`, divididos por responsabilidade e não por tela:

Pasta	Linhas	O que vive ali
`src/api/`	1.314	tipos do contrato, cliente HTTP e uma função por endpoint
`src/adapters/`	1.871	tradução entre o formato da tela e o da API
`src/screens/`	3.555	as telas e suas janelas de operação
`src/hooks/`	75	`useRecurso`: carregando, erro e dados de uma busca
`src/ui/`	39	os três avisos: carregando, vazio e erro
`src/test-utils/`	1.093	a API falsa usada pelos testes
raiz de `src/`	1.926	`App.tsx`, `data.ts`, `components.tsx`, `RecordForm.tsx`

A regra que organiza isso é simples de enunciar e foi seguida sem exceção: **nenhuma tela chama `fetch`, e nenhuma tela monta o corpo de uma requisição**. A tela chama uma função de `src/api/`, passando pelo adapter. Quem quiser saber o que vai pela rede olha em dois lugares, não em quinze.

As dependências de produção são três: `react`, `react-dom` e `@fontsource/inter` (a fonte, servida localmente em vez de buscada no Google Fonts). Não há biblioteca de requisição, de formulário, de estado ou de componentes. Tudo o que o sistema faz está escrito no repositório.

2.2 TypeScript sem `any`

O enunciado proíbe `any`, e a proibição é mais interessante do que parece: ela aparece exatamente nos pontos em que dados de fora entram no programa. Dois casos concretos:

O corpo de uma resposta de erro. `response.json()` devolve algo que o compilador não conhece. Em vez de afirmar que é um erro da API, o código *pergunta*:

```
function pareceCorpoDeErro(valor: unknown): valor is CorpoDeErro {
  if (typeof valor !== 'object' || valor === null) return false;
  const possivel = valor as Partial<CorpoDeErro>;
  return typeof possivel.mensagem === 'string'
      && typeof possivel.status === 'number';
}
```

O tipo `valor is CorpoDeErro` é um *type guard*: depois de um `if (pareceCorpoDeErro(conteudo))`, o compilador passa a tratar `conteudo` como `CorpoDeErro` dentro daquele bloco. É a diferença entre verificar e simplesmente alegar.

O endereço da API. O caminho natural seria `import.meta.env.VITE_API_URL`, mas o tipo de `import.meta.env` inclui `[key: string]: any` — ler qualquer variável de ambiente traz um `any` junto.

Como o endereço é fixo e documentado, virou uma constante comum:

```
export const BASE_URL = 'http://localhost:8080/api';
```

Perde-se a configuração por ambiente, que este trabalho não usa; ganha-se um arquivo sem `any` e uma linha que qualquer pessoa do grupo sabe onde trocar.

2.3 Um formulário que se monta a partir de um JSON

São onze formulários de cadastro, todos com a mesma aparência e as mesmas regras: campos agrupados em linhas, separadores pretos, asterisco no obrigatório, máscara em dinheiro e data. Escrever onze componentes seria onze vezes a mesma coisa — e onze lugares para esquecer de corrigir um defeito.

Em vez disso, o agrupamento dos campos extraído do Figma virou dados, em `src/form-layouts.json`, e um único componente (`RecordForm.tsx`, 301 linhas) desenha qualquer um deles. O tipo das telas sai do próprio JSON:

```
export type Entity = keyof typeof formLayouts;
```

Essa linha faz mais do que parece. Se alguém acrescentar uma tela ao JSON sem configurá-la em `data.ts`, o compilador acusa. E o teste que percorre `Object.keys(layouts)` passa a cobrir a tela nova automaticamente, sem ninguém lembrar de incluí-la.

2.4 Duas telas que servem quatro cadastros

No livro, Clientes e Fornecedores são dois cadastros; Pagamentos e Recebimentos, duas telas. Na API, cada par é a mesma coisa vista de dois ângulos: cliente e fornecedor são a mesma `Pessoa` com `papel` diferente, e os dois títulos têm endpoints espelhados.

Duas telas atendem os quatro cadastros, parametrizadas:

- `PessoasScreen` recebe o `Papel` — `CLIENTE` ou `FORNECEDOR` — e muda o título, as colunas e o filtro.
- `TitulosScreen` recebe uma `ConfiguracaoDoLado` (os rótulos: "Fornecedor" ou "Cliente", "Quitar" ou "Receber") e um objeto `OperacoesDeTitulo` com as funções da API daquele lado.

O segundo caso é o mais ilustrativo. A tela não sabe se está lidando com contas a pagar ou a receber: ela recebe as operações prontas e as chama. Quem decide é `operacoes-de-titulo.ts`, que liga cada lado à sua API. É o mesmo princípio da injeção de dependência, escrito sem framework nenhum.

2.5 Estado: o que a tela guarda e o que ela busca

Não há biblioteca de estado global. Cada tela guarda apenas três coisas: qual registro está selecionado, qual janela está aberta, e o que o usuário digitou na busca. **Os dados em si nunca são guardados** — eles vêm do `useRecurso` e são buscados de novo depois de cada gravação.

Isso é uma decisão, não um descuido. Guardar a lista em memória e atualizá-la à mão depois de um POST cria duas verdades: a do servidor e a da tela. Quando a API calcula algo que o front-end não calcula —

saldo, dias de atraso, situação do título — a cópia local envelhece e passa a mentir. Buscar de novo custa uma requisição e elimina a classe inteira de defeitos.

> **Nota histórica.** A primeira versão do front-end guardava tudo em `localStorage`, porque a API ainda não existia. Conforme as telas foram integradas, essa camada ficou sem uso — e foi removida inteira, 871 linhas, num commit só. Código morto que ninguém apaga vira a primeira coisa que alguém lê por engano na hora de explicar o sistema.

2.6 Testes

São **120 testes** em cinco arquivos, rodando em Vitest com Testing Library:

Arquivo	Testes	O que cobre
`App.test.tsx`	24	navegação, formulários de todas as telas, busca, operações
`adapters/pessoas.test.ts`	47	a tradução mais complexa: papéis, telefones, endereços
`adapters/plano-contas.test.ts`	16	árvore recursiva e contas analíticas
`adapters/titulos.test.ts`	13	os dois lados do caixa e os rateios
`adapters/despesas.test.ts`	11	previsão de gasto e geração de título

Os testes de tela não falam com o servidor: `src/test-utils/apiFalsa.ts` substitui `fetch` por uma API em memória, com 1.093 linhas. Ela não é um *mock* que devolve sempre o mesmo objeto — reproduz as regras do back-end, **inclusive as recusas**: 422 ao cancelar título já pago, 409 ao excluir banco com agência, 422 ao lançar em conta consolidada. É o que permite testar o caminho de erro, que é justamente o que ninguém testa à mão.

3. Integração com a API

Esta é a seção central do trabalho. O enunciado exige *pelo menos uma* funcionalidade integrada à API REST; o resultado entregue tem **as doze telas de cadastro consumindo a API**, sem nenhuma dependência de armazenamento no navegador.

3.1 O caminho de uma requisição

Toda interação com o servidor percorre a mesma sequência de quatro camadas:

```
Tela (vocabulário do livro)
   ↓  adapter: registroParaPessoa()
Corpo no formato do contrato
   ↓  src/api/pessoas.ts: criarPessoa()
Cliente HTTP: requisitar<PessoaApi>('/pessoas', { metodo: 'POST', corpo })
   ↓  fetch
API REST (Spring Boot)
```

E a volta é a mesma sequência invertida: o JSON vira `PessoaApi`, o adapter transforma em `RecordData`, e a tabela desenha. Cada camada tem uma responsabilidade que cabe numa frase, o que torna fácil dizer onde procurar quando algo dá errado.

3.2 O cliente HTTP

Um arquivo, `src/api/http.ts`, concentra tudo que é HTTP. Ele faz três coisas: monta a URL, envia e recebe JSON, e transforma os erros do contrato em exceções tipadas.

O contrato define um formato único de erro para os status 400, 404, 409 e 422, com uma `mensagem` já em português e legível para o usuário final. Isso permitiu uma decisão que simplificou muito as telas: **a mensagem da API é exibida como está**, sem tabela de tradução no front-end. Quando o servidor diz "*Titulo com pagamento registrado nao pode ser cancelado*", é isso que aparece na tela.

Há duas classes de erro, e a distinção importa para o usuário:

- `ErroApi` — o servidor respondeu e recusou. Carrega `status`, `codigo` (por exemplo `REGRA_NEGOCIO`) e a lista de `campos` rejeitados nos erros de validação.
- `ErroDeRede` — o navegador não conseguiu falar com o servidor. A mensagem já sugere a causa provável: "*Verifique se a API está rodando em http://localhost:8080/api*".

A diferença é concreta: no primeiro caso o usuário precisa corrigir o que digitou; no segundo, precisa subir o back-end. Um erro genérico não diz qual dos dois é.

3.3 O padrão Adapter

O problema que o padrão resolve aqui é literal: duas interfaces incompatíveis que precisam conversar sem que nenhuma das duas mude.

A tela diz	A API diz
Registro	`id`
Cliente / Fornecedor	`nome` + `papeis`
CPF / CNPJ	`documento`
Endereço (uma linha)	`logradouro`, `numero`, `complemento`
"R$ 1.500,00"	`1500.00`
"15/11/2026"	`"2026-11-15"`

A tela trabalha com um tipo deliberadamente simples:

```
export type RecordData = { id: string; [key: string]: string };
```

Tudo é texto, porque tudo o que um formulário HTML produz é texto. Número, data e enumeração **só existem do lado da API**, e a conversão acontece em um lugar: o adapter. Cada um expõe duas funções simétricas — por exemplo `pessoaParaRegistro()` e `registroParaPessoa()`.

O ganho prático apareceu quando o back-end mudou. Quando a API ganhou quatro campos novos em `Pessoa` (condição de pagamento, desconto negociado, prazo de entrega, vendedor), a alteração foi no adapter e no JSON de layout. As telas não foram tocadas. É exatamente isso que o padrão promete, e foi verificado na prática.

3.4 O hook de carregamento

Toda tela integrada precisa das mesmas três informações: se está carregando, se deu erro, e quais são os dados. Em vez de repetir esse controle doze vezes, ele está em `useRecurso<T>`, com 75 linhas.

```
const recurso = useRecurso(
  () => listarPessoas(papel, busca),
  `pessoas:${papel}:${busca}`,
);
```

O segundo argumento é uma **chave descritiva**, e não a lista de dependências habitual do `useEffect`. A escolha foi deliberada: a chave diz, em texto legível, o que provoca uma nova busca. Quem lê `'pessoas:CLIENTE:erica'` entende na hora; quem lê `[papel, busca]` precisa procurar onde essas variáveis são usadas.

Duas sutilezas de React que o hook resolve, e que valem ser explicadas:

- A função `buscar` é recriada a cada *render*. Se ela entrasse na lista de dependências, o efeito rodaria sem parar. Ela é guardada numa `useRef`, de onde o efeito lê sempre a versão mais recente sem depender dela.
- Se o usuário digitar na busca enquanto uma resposta antiga está a caminho, a resposta antiga pode chegar depois da nova. A função de limpeza marca `cancelado = true`, e o resultado tardio é descartado em vez de sobrescrever os dados corretos.

3.5 O problema que mais deu trabalho: PUT substitui a coleção

Este foi o defeito mais instrutivo do projeto, e vale descrevê-lo inteiro, porque ele não aparece em teste de tela nenhum e não dá erro.

No contrato, `PUT /api/pessoas/{id}` **substitui** as coleções que recebe. Se o corpo trouxer um telefone, a pessoa passa a ter um telefone — os outros somem. O formulário do livro mostra um telefone e um endereço; a pessoa, no banco, pode ter vários.

O resultado: editar o nome de um cliente apagava silenciosamente o FAX dele. A tela mostrava "Cliente atualizado", a requisição retornava 200, e um dado tinha sido perdido.

A correção foi dar ao adapter acesso ao registro atual, para ele devolver o que o formulário não mostra:

```
registroParaPessoa(valores, papel, pessoaAtual?)
```

O mesmo problema apareceu em três outros lugares — papéis, endereços e os rateios de um título — e a mesma correção se aplicou. O caso dos papéis era o mais grave: uma pessoa que fosse cliente *e* fornecedor perdia um dos dois papéis ao ser editada por qualquer uma das duas telas.

Houve ainda um agravante que custou a aparecer. O primeiro conserto lia `telefones[0]`, supondo que o primeiro item fosse o exibido. Mas o back-end devolve um `Set` sem ordem garantida, e em alguns registros o FAX vinha primeiro — a tela mostrava o FAX no campo "Telefone" e, ao salvar, gravava um no lugar do outro. A correção final seleciona pelo **tipo**, não pela posição, e usa a mesma função na leitura e na gravação:

```
export function indiceDoTelefoneExibido(telefones: { tipo: string }[]): number {
  const comercial = telefones.findIndex(item => item.tipo === 'COMERCIAL');
  return comercial === -1 ? (telefones.length ? 0 : -1) : comercial;
}
```

Usar a mesma função nos dois sentidos é o que garante que a tela grave exatamente o item que mostrou. São 47 testes no adapter de pessoas — a maior concentração do projeto — e a razão é esta.

3.6 Regras de negócio na interface

O back-end valida tudo; a questão é *quando* o usuário descobre. A política adotada tem três casos, e a diferença entre eles é se o navegador tem como saber a resposta sozinho:

Caso	Quem responde	Exemplo
O navegador tem os dados	a tela barra antes de chamar a API	RN12: a soma do rateio precisa bater com o valor do título
A tela pode evitar a escolha inválida	o formulário só oferece o que é válido	RN09: só título vencido e em aberto vai a protesto — a lista vem de `/contas-receber/inadimplentes`
Só o servidor sabe	a API recusa e a tela mostra a mensagem dela	RN11: título com pagamento registrado não pode ser cancelado

O segundo caso é o mais interessante de defender. No ciclo do cheque (`EM_CARTEIRA → DEPOSITADO → COMPENSADO` ou `DEVOLVIDO`), a janela oferece **apenas os destinos válidos a partir de onde o cheque está**. Um cheque compensado não mostra botão de confirmar. A alternativa — aceitar qualquer destino e deixar a API recusar com 422 — funcionaria, mas transformaria a interface num jogo de adivinhação.

3.7 Como a integração foi validada

Duas camadas de verificação, com propósitos diferentes.

Automática. Os 120 testes rodam contra a API falsa e cobrem o caminho feliz e o de erro. São rápidos e rodam a cada alteração.

Manual, contra o back-end real. Cada operação foi exercida no navegador e o efeito conferido **direto na API**, não apenas na tela. A distinção é importante: a tela mostrar "Título cancelado" não prova que o título foi cancelado. Alguns exemplos do registro de validação:

Operação	Conferência
Cancelar o título `TESTE-FRONT-01`	`GET /contas-pagar` passou a trazer `situacao: CANCELADA`, com a linha ainda na lista
Cancelar `DUP-123456`, que está `PAGA`	a tela exibiu a recusa da API (RN11) e o título continuou `PAGA`
Inativar `Cliente Teste Campos SA`	`GET /pessoas?papel=CLIENTE` passou a trazer `ativo: false`, sem perder os títulos
Excluir um banco que tem agência	409 com a mensagem do servidor, exibida na tela
Filtro "Só os a depositar"	com dois cheques em carteira, um vencido e um não, a lista trouxe apenas o vencido

Foi essa validação manual que encontrou os defeitos descritos em 3.5. Nenhum deles produzia erro: a requisição retornava 200 e a tela dizia que tinha dado certo. Só comparando o que a API guardou com o que o usuário via é que a perda de dado aparecia.

> **Defeitos relatados ao back-end.** O front-end identificou cinco problemas do lado do servidor durante a integração, entre eles um 409 ao editar título com rateio (conflito de chave única na ordem de *insert* e *delete* do Hibernate) e filtros de nome e papel que eram mutuamente exclusivos. Todos foram repassados e corrigidos, e os contornos provisórios do front-end foram removidos depois.

4. Uso de inteligência artificial

O enunciado exige documentar o uso de IA. Esta seção resume o que foi feito; o registro completo — prompts, decisões e desafios, em quinze seções — está em `frontend/docs/RELATORIO-IA.md`, versionado junto com o código.

4.1 Como a IA foi usada

A ferramenta foi o Claude Code, usado como par de programação ao longo de quinze sessões de trabalho. O padrão de uso foi sempre o mesmo: descrever o objetivo e as restrições, receber um plano, aprovar ou corrigir o plano, e só então deixar escrever código.

As restrições dadas no início valeram para o projeto inteiro:

- React, TypeScript e CSS puro; sem Tailwind e sem biblioteca de componentes;
- `any` proibido;
- o padrão Adapter como fronteira entre a tela e a API;
- nada de alterar o `backend/` — o que faltasse lá seria pedido ao responsável;
- "*Teste no navegador o que escrever. Não entregue código não executado.*"
- "*Prefira código legível a código curto.*"

As duas últimas foram as que mais mudaram o resultado, e não por acaso: são as que vão contra a tendência natural da ferramenta.

4.2 O que ela acertou

O ganho maior foi em trabalho repetitivo com forma bem definida: transcrever o contrato da API para tipos TypeScript (as 600 linhas de `src/api/tipos.ts`), escrever os pares de função dos adapters, montar a API falsa dos testes, e aplicar uma mesma correção em quatro lugares parecidos.

Também foi útil como revisora. A varredura que encontrou os combos com nomes fixos herdados da versão em `localStorage` — opções que não correspondiam a conta nenhuma e dariam erro na gravação — saiu de um pedido explícito de revisão geral antes da entrega.

4.3 O que ela errou, e como os erros apareceram

Esta parte é a mais importante do relatório, porque descreve o que o grupo precisou fazer *além* de pedir.

Perda silenciosa de dados. A primeira versão do adapter de pessoas enviava no PUT apenas o que o formulário mostrava. Como o contrato diz que o PUT substitui a coleção, editar um cliente apagava o FAX, e editar quem era cliente e fornecedor apagava um dos papéis. O código parecia correto, compilava, passava nos testes que existiam e retornava 200. Só apareceu quando alguém comparou, registro a registro, o que a API tinha guardado com o que havia antes.

Uma correção pela metade. O primeiro conserto desse defeito lia `telefones[0]`. Funcionava nos registros de teste e falhava nos registros em que o FAX vinha primeiro — porque o back-end devolve um `Set`, sem ordem garantida. A versão correta seleciona pelo tipo.

Verificação que não verificava. Durante várias sessões, a checagem de tipos foi reportada como limpa quando não era. O comando usado era `npx tsc -b | head -4 && echo "TYPECHECK OK"` — em um *pipeline*, o shell devolve o código de saída do último comando, que é o `head`, e esse sempre dá zero. Passou a ser `npx tsc -b > log 2>&1; echo "exit=$?"`.

Entregas declaradas prontas com lacunas. Mais de uma vez o trabalho foi dado por concluído enquanto faltavam operações — telas sem o botão de cancelar, de inativar, de excluir, embora os endpoints já existissem no contrato. Foi preciso cobrar explicitamente que tudo fosse fechado antes de seguir. A varredura final comparou, tela a tela, a barra de botões com a lista de endpoints do contrato; foi assim que as cinco lacunas da última etapa apareceram.

4.4 O método que sobrou

Os erros acima têm uma coisa em comum: nenhum deles aparece lendo o código. Todos aparecem comparando o que o programa *diz* que fez com o que o servidor *guardou*. Daí as três práticas que acabaram valendo como método:

1. **Conferir o efeito, não a mensagem.** Depois de cada operação, consultar a API e olhar o registro. "Salvo com sucesso" não é evidência de nada.
2. **Desconfiar do comando de verificação.** Um comando que sempre diz "OK" provavelmente não está verificando. Vale testar o verificador reintroduzindo o defeito de propósito — foi o que se fez com o teste do rateio, que falhou como devia antes de a correção ser restaurada.
3. **Comparar com a especificação, não com a impressão.** A lista de endpoints do contrato, confrontada com as telas, encontrou lacunas que a leitura das telas não encontrava.

A conclusão honesta é que a IA acelerou bastante a escrita e quase nada a verificação. Os defeitos mais caros do projeto foram todos de uma espécie que a ferramenta não detecta sozinha — código plausível, que compila, que passa nos testes existentes e que está errado. O tempo economizado na digitação foi, em boa parte, reinvestido em conferência.

Anexo A — Como executar

O sistema tem duas partes, que sobem separadamente.

Back-end (API e banco)

```
cd backend
docker compose up -d
mvnw.cmd spring-boot:run
```

A API sobe em `http://localhost:8080` e o CORS já libera as portas 5173 e 3000.

Front-end

```
cd frontend
npm install
npm run dev
```

Requer Node.js 22.12 ou superior. O menu inicial fica em `/#/inicio`.

Verificações

```
npm test                        # 120 testes
npm run build                   # checagem de tipos e build de produção
node scripts/verify-assets.mjs  # confere os 163 SVGs do protótipo
```

Com a API fora do ar, as telas mostram um aviso com a opção de tentar de novo — o estado de erro é parte da interface, não uma tela branca.

Anexo B — Telas e endpoints

As doze telas de cadastro, o endpoint que cada uma consome e as operações disponíveis. Nenhuma depende de armazenamento no navegador.

Tela	Endpoint	Operações
Clientes	`/api/pessoas?papel=CLIENTE`	listar, buscar, incluir, editar, inativar
Fornecedores	`/api/pessoas?papel=FORNECEDOR`	listar, buscar, incluir, editar, inativar
Pagamentos	`/api/contas-pagar`	lançar, editar, quitar, cancelar, localizar
Recebimentos	`/api/contas-receber`	lançar, editar, receber, cancelar, localizar
Plano de Contas	`/api/plano-contas`	árvore, incluir, editar, excluir folha
Despesas	`/api/despesas`	incluir, editar, excluir, gerar título
Bancos	`/api/bancos`	árvore banco › agência › conta, com inclusão e exclusão em cada nível
Cartórios	`/api/cartorios`	incluir, editar, excluir
Lançamentos	`/api/lancamentos`	partida dobrada, com estorno
Cheque Pré-Datado	`/api/cheques`	incluir, editar, movimentar o ciclo, excluir, filtrar os a depositar
Remessa p/ Cartório	`/api/remessas-cartorio`	enviar a protesto, protestar, baixar
Relatórios	`/api/relatorios`	resumo do período, por categoria, inadimplência

Baixas: o que cada botão faz de verdade

Nos títulos, nas pessoas e nos lançamentos, o `DELETE` da API **não apaga o registro** — ele muda a situação, porque o histórico depende de a linha continuar existindo. Os rótulos dos botões foram escolhidos para dizer isso:

Botão	Tela	Efeito
Cancelar	Pagamentos, Recebimentos	situação vira `CANCELADA`, a linha fica na lista
Inativar	Clientes, Fornecedores	fica `ativo = false`, os títulos continuam válidos
Excluir	Lançamentos	vira `ESTORNADO`, o lançamento fica na lista
Exclusão	Plano de Contas	aí sim a categoria some, se não tiver subcontas

Chamar os três primeiros de "Excluir" ensinaria o usuário errado. Cada janela de confirmação explica o que vai acontecer de fato — "*Ele continuará na lista, com a situação CANCELADA*", "*Ele não é excluído, porque os títulos já lançados dependem dele*".

> **Limites conhecidos.** Não há autenticação, conciliação bancária nem integração com instituição financeira; nenhuma ação executa pagamento de verdade. O endereço da API é fixo em `src/api/http.ts`, sem configuração por ambiente. Dos 17 capítulos de referência do livro, estão cobertos todos os que têm endpoint no contrato.